COMPILER DESIGN LABORATORY • REFERENCE MANUAL

Laboratory Algorithms: Lex, Yacc & Code Optimization

Formal operational logic and procedural steps for core compiler construction experiments.

1. Lexical Analyzer using Lex Tool

`lexical.l`

1. **Start** the lexical scanning process and initialize input buffers.
2. Specify regular expression pattern specifications in the definition section:
   - **Keywords:** `"int"|"float"|"char"|"if"|"else"|"while"|"return"`
   - **Identifiers:** `[a-zA-Z_][a-zA-Z0-9_]*`
   - **Constants:** Numeric sequences `[0-9]+`
   - **Operators:** Arithmetic, assignment & relational tokens (+, -, *, /, =, <, >, ==, <=, >=, !=)
   - **Delimiters:** Parentheses, braces, semicolons, commas (`(`), `{`, `;`, `,`)
   - **Whitespace:** Tabs, spaces, and newline characters (suppress output action)
   - **Unrecognized:** Fallback rule matching any remaining characters
3. Invoke `yylex()` from `main()` to scan input from standard stream (`stdin`).
4. Apply the maximal munch principle (longest matching token rule) for lexeme resolution.
5. Print each lexeme alongside its corresponding classified token attribute.
6. Upon reading End-of-File (EOF), execute `yywrap()` returning 1 to close input.
7. **Stop**.

2. Count Lines, Words, and Characters

`count.l`

1. **Start** and initialize integer accumulator counters: `lines = 0`, `words = 0`, `chars = 0`.
2. Define Lex transition pattern rules:
   - **Rule 1 (`\n`):** Increment `lines` by 1; increment `chars` by 1.
   - **Rule 2 (`[ \t]+`):** Add matched lexeme length (`yyleng`) to `chars`.
   - **Rule 3 (`[^ \t\n]+`):** Increment `words` by 1; add matched token length (`yyleng`) to `chars`.
3. Invoke `yylex()` within `main()` to initiate stream evaluation from console.
4. Continuously update tally registers as matching character sequences are encountered.
5. On receiving stream termination signal (EOF / Ctrl+D), complete scanning.
6. Display the accumulated values of total Lines, Words, and Characters.
7. **Stop**.

3. Substring Substitution (abc to ABC)

`convertABC.l`

1. **Start** the lexical pattern replacement scanner.
2. Define the pattern replacement rules in the Lex rule section:
   - **Target Rule (`abc`):** Direct match on literal sequence `abc`; execute `printf("ABC")`.
   - **Default Rule (`. | \n`):** Match any other single character or newline; trigger `ECHO` to output unaltered.
3. Prompt user for raw input string within the `main()` routine.
4. Initiate pattern evaluation by executing `yylex()`.
5. Stream altered and unaltered characters directly to the standard output buffer.
6. Terminate execution upon encountering End-of-File (EOF).
7. **Stop**.

4. Recognition of Valid Arithmetic Expressions

`arith_expr.l • arith_expr.y`

A. LEXICAL PHASE (SCANNER ALGORITHM)

1. Scan numeric sequences `[0-9]+`, convert string via `atoi(yytext)` into `yylval`, and return token `NUMBER`.
2. Filter and ignore whitespace characters (spaces and horizontal tabs `[ \t]`).
3. On encountering newline (`\n`), return `'\n'` as expression delimiter.
4. For operators and parentheses (`+`, `-`, `*`, `/`, `(`, `)`), return raw character ASCII code.

B. PARSING PHASE (YACC LALR(1) ALGORITHM)

1. Declare terminal token `NUMBER` with operator precedence and associativity:
   - `%left '+' '-'` (lowest precedence, left associative)
   - `%left '*' '/'` (higher precedence, left associative)
   - `%left UMINUS` (highest precedence for unary negation)
2. Formulate context-free grammar production rules:
   - `stmt → expr '\n'` (Action: print *"Valid Arithmetic Expression"*)
   - `expr → expr '+' expr | expr '-' expr | expr '*' expr | expr '/' expr`
   - `expr → '-' expr %prec UMINUS | '(' expr ')' | NUMBER`
3. Invoke `yyparse()` from `main()` to construct derivation tree.
4. Shift terminals and reduce handles according to LALR(1) parsing table.
5. If input syntax is malformed, divert to `yyerror()` and print *"Syntax Error"*.
6. **Stop**.

5. Recognition of Valid Identifiers

`identifier.l • identifier.y`

A. LEXICAL PHASE (SCANNER ALGORITHM)

1. Scan valid identifier pattern `[a-zA-Z][a-zA-Z0-9]*` (starts with an alphabet followed by alphanumeric characters) and return token `ID`.
2. Scan newline character `\n` and return `'\n'` as statement terminator.
3. Match any invalid isolated character (`.`) and return its literal value to trigger parser error checking.

B. PARSING PHASE (YACC SYNTAX GRAMMAR)

1. Declare terminal token `ID`.
2. Define verification grammar productions:
   - `input → ID '\n'` (Action: print *"Valid Identifier"* and return 0)
   - `input → error '\n'` (Action: print *"Invalid Identifier"* and return 0)
3. Execute `yyparse()` inside driver function `main()`.
4. If string adheres to identifier formation rules, reduce to `input → ID '\n'`.
5. If invalid leading characters (digits, illegal punctuation) are supplied, parser recovers via `error` token and reports rejection.
6. **Stop**.

6. Computation of FIRST and FOLLOW Sets

`firstAndFollow.c`

A. ALGORITHM FOR FIRST(X)

1. If symbol X is a terminal, then `FIRST(X) = {X}`.
2. If production $X \to \$$ exists (where $ denotes ε), add $ to `FIRST(X)`.
3. If X is a non-terminal and production $X \to Y_1\, Y_2 \ldots Y_k$ exists:
   - Add all non-ε symbols from $\text{FIRST}(Y_1)$ to `FIRST(X)`.
   - If $\text{FIRST}(Y_1)$ contains ε, iteratively evaluate $\text{FIRST}(Y_2)$ through $\text{FIRST}(Y_k)$ until a symbol without ε is reached.
   - If ε belongs to FIRST of all RHS symbols $Y_1 \ldots Y_k$, add $ to `FIRST(X)`.

B. ALGORITHM FOR FOLLOW(A)

1. For grammar start symbol S, insert End-of-Input marker $ into `FOLLOW(S)`.
2. For every production of form $A \to \alpha B \beta$ (where non-terminal B is on RHS):
   - Add all symbols from $\text{FIRST}(\beta)$ excluding ε into `FOLLOW(B)`.
3. For productions $A \to \alpha B$ or $A \to \alpha B \beta$ where $\text{FIRST}(\beta)$ derives ε:
   - Add all symbols in `FOLLOW(A)` into `FOLLOW(B)`.
4. Eliminate duplicate entries across sets; return resultant collections.

7. Constant Propagation Optimization

`constant_propagation.c`

1. **Start** and read total count of three-address quadruples n.
2. Store each instruction as a record: op, `arg1`, `arg2`, `res`; set `flag = 0`.
3. Traverse each quadruple statement i from index 0 to n-1:
   - Check if statement is constant assignment: op `==` `"="` and `isNum(arg1)`.
   - If true, copy constant `val = arg1` and identifier `var = res`.
   - Mark instruction as redundant by updating `flag = 1`.
   - Scan subsequent instructions $j = i + 1$ to n-1:
     - If `arr[j].arg1 == var`, replace `arg1` with `val`.
     - If `arr[j].arg2 == var`, replace `arg2` with `val`.
     - If `arr[j].res == var`, variable is reassigned; break forward loop.
4. Output only active quadruples (`flag == 0`) representing optimized code.
5. **Stop**.

8. Intermediate Code Generation to Target Assembly

`intermediate.c`

1. **Start**; open intermediate representation `input.txt` in read mode and `output.txt` in write mode.
2. Verify file handles are non-null; abort with error notification if either file cannot be accessed.
3. Read quadruples sequentially: `fscanf(fp1, "%s%s%s%s", op, arg1, arg2, res) == 4`.
4. Emit assembly instructions via accumulator register R0:
   - `"+"` → `MOV R0, arg1; ADD R0, arg2; MOV res, R0`
   - `"-"` → `MOV R0, arg1; SUB R0, arg2; MOV res, R0`
   - `"*"` → `MOV R0, arg1; MUL R0, arg2; MOV res, R0`
   - `"/"` → `MOV R0, arg1; DIV R0, arg2; MOV res, R0`
   - `"="` → `MOV R0, arg1; MOV res, R0`
5. Close file pointers `fp1` and `fp2` after reaching end of stream.
6. **Stop**.